

```
// =====  
// FILE: docs/Diagramma_ER_V4.0_Fase2.dbml  
// VERSIONE: ER V4.0 - FASE 2  
// TARGET: 26 TABELLE - GERARCHIE SERVIZI E ASSET HARDENIZZATE  
// DESCRIZIONE:  
// - Allineamento con gli script produttivi 01-14  
// - Organizzazione obbligatoria per asset e servizi  
// - Categoria ACN obbligatoria per gli asset  
// - Vincoli UNIQUE sulle tabelle di dominio  
// - Codici applicativi stabili per servizi e asset  
// - Gerarchia servizio-sottoservizio storicizzabile e aciclica  
// - Gerarchia asset-sotto-asset storicizzabile e aciclica  
// - Audit log senza FK intenzionale verso asset  
// =====
```

```
// -----  
// CORE E ANAGRAFICHE  
// -----
```

```
Table organizzazione {  
    id serial [pk]  
    nome varchar [not null]  
    descrizione text  
}
```

```
Table categoria_asset {  
    id serial [pk]  
    codice_acn varchar [unique, note: 'Codice tassonomia ACN, ad esempio AC:IN_HW-CS_SR']  
    nome varchar [not null]  
    descrizione text  
}
```

```
Table tipo_servizio {  
  id serial [pk]  
  nome varchar [not null, unique]  
  descrizione text  
}
```

```
Table tipo_fornitore {  
  id serial [pk]  
  nome varchar [not null, unique]  
  descrizione text  
}
```

```
Table ruolo_organigramma {  
  id serial [pk]  
  nome varchar [not null, unique]  
  descrizione text  
}
```

```
Table responsabile {  
  id serial [pk]  
  nome varchar  
  cognome varchar  
  email varchar [unique]  
  telefono varchar  
  organizzazione_id int [ref: > organizzazione.id]  
  ruolo_organigramma_id int [ref: > ruolo_organigramma.id]  
}
```

```
Table ruolo {  
  id serial [pk]  
  nome varchar [unique]
```

```
    descrizione text
```

```
}
```

```
Table responsabile_ruolo {
```

```
    responsabile_id int [pk, ref: > responsabile.id]
```

```
    ruolo_id int [pk, ref: > ruolo.id]
```

```
    is_titolare boolean [default: true]
```

```
}
```

```
// -----
```

```
// VULNERABILITA
```

```
// -----
```

```
Table vulnerabilita {
```

```
    id serial [pk]
```

```
    codice_bollettino varchar [unique, note: 'Riferimento ACN o CSIRT']
```

```
    descrizione_rischio text
```

```
    livello_severita varchar [note: 'Informativa, Bassa, Media, Alta, Critica']
```

```
    data_pubblicazione date [default: `now()`]
```

```
}
```

```
Table asset_vulnerabilita {
```

```
    asset_id int [pk, ref: > asset.id]
```

```
    vulnerabilita_id int [pk, ref: > vulnerabilita.id]
```

```
    data_rilevamento timestamp [default: `now()`]
```

```
    stato_remediation varchar [note: 'OPEN, PATCHED, MITIGATED']
```

```
}
```

```
// -----
```

```
// ASSET, SERVIZI E FORNITORI
```

```
// -----
```

```
Table asset {
```

```
  id serial [pk]
```

```
  codice_asset varchar(80) [
```

```
    not null,
```

```
    unique,
```

```
    note: 'Codice stabile per GUI, API e import/export XLS'
```

```
]
```

```
  nome varchar [not null]
```

```
  categoria_asset_id int [
```

```
    not null,
```

```
    ref: > categoria_asset.id
```

```
]
```

```
  classificazione_criticita varchar [
```

```
    note: 'Bassa, Media, Alta, Critica'
```

```
]
```

```
  descrizione text
```

```
  ubicazione varchar
```

```
  versione varchar
```

```
  data_inserimento date [default: `now()`]
```

```
  organizzazione_id int [
```

```
    not null,
```

```
    ref: > organizzazione.id
```

```
]
```

responsabile_id int [ref: > responsabile.id]

Note: '''

Ogni asset deve appartenere a un'organizzazione e deve essere associato a una categoria ACN.

codice_asset rispetta il formato:

^[A-Z0-9][A-Z0-9_-]{2,79}\$

La composizione tra asset e sotto-asset è rappresentata dalla tabella asset_componente.

'''

}

Table fornitore {

id serial [pk]

nome varchar [not null]

tipo_fornitore_id int [ref: > tipo_fornitore.id]

indirizzo varchar

contatto_email varchar

Note: '''

La relazione fornitore-subfornitore e il codice stabile del fornitore saranno introdotti nelle fasi successive dell'ER V4.0.

'''

}

Table servizio {

id serial [pk]

codice_servizio varchar(80) [

not null,

```
unique,  
note: 'Codice stabile per GUI, API e import/export XLS'  
]
```

```
nome varchar [not null]  
descrizione text  
tipo_servizio_id int [ref: > tipo_servizio.id]  
stato_servizio_id int [ref: > stato_servizio.id]
```

```
organizzazione_id int [  
    not null,  
    ref: > organizzazione.id  
]
```

```
responsabile_id int [ref: > responsabile.id]
```

```
Note: ""  
codice_servizio rispetta il formato:  
^[A-Z0-9][A-Z0-9_-]{2,79}$
```

La composizione tra servizi e sottoservizi è rappresentata
dalla tabella servizio_componente.

```
""  
}
```

```
// -----  
// COMPOSIZIONE GERARCHICA DEGLI ASSET  
// -----
```

```
Table tipo_relazione_asset {  
    id int [pk, increment]
```

```
codice varchar(50) [  
    not null,  
    unique,  
    note: 'Codice applicativo della tipologia di relazione'  
]
```

```
nome varchar(100) [not null, unique]  
descrizione text
```

Note: '''

Dominio normalizzato utilizzato per classificare semanticamente
le relazioni tra asset e sotto-asset.

Il codice rispetta il formato:

`^[A-Z0-9][A-Z0-9_]{2,49}$`

Esempi iniziali:

- COMPOSIZIONE_FISICA;
- COMPONENTE_LOGICO;
- VIRTUALIZZAZIONE;
- CLUSTER;
- DIPENDENZA_CONDIVISA;
- ALTRO.

'''

```
}
```

```
Table asset_componente {  
    id bigint [pk, increment]
```

```
    asset_padre_id int [  
        not null,
```

ref: > asset.id

]

asset_figlio_id int [

not null,

ref: > asset.id

]

tipo_relazione_asset_id int [

not null,

ref: > tipo_relazione_asset.id

]

descrizione text

attiva boolean [

not null,

default: true

]

valido_dal timestampz [

not null,

default: `now()`

]

valido_al timestampz

relazione_primaria boolean [

not null,

default: false

]


```
ordine_componente int [  
    not null,  
    default: 1  
]
```

motivo_chiusura text

```
creato_il timestamptz [  
    not null,  
    default: `now()`  
]
```

```
aggiornato_il timestamptz [  
    not null,  
    default: `now()`  
]
```

Note: ""

Relazione direzionale:

asset_padre_id -> asset_figlio_id.

Il modello consente:

- più sotto-asset per uno stesso asset padre;
- più asset padre per un componente condiviso;
- un solo eventuale padre primario attivo per ogni sotto-asset;
- gerarchie multilivello;
- storicizzazione delle relazioni.

Regole implementate in PostgreSQL:

- padre e figlio devono essere differenti;
- ordine_componente deve essere maggiore di zero;
- valido_al non può precedere valido_dal;

- una relazione attiva non può avere valido_al;
- una relazione chiusa deve avere valido_al;
- una relazione primaria deve essere attiva;
- una relazione chiusa non può essere riattivata;
- una sola relazione attiva per ciascuna coppia padre-figlio;
- un solo padre primario attivo per ogni sotto-asset;
- trigger trg_check_asset_componente_cycle contro i cicli;
- FK padre, figlio e tipologia con ON DELETE RESTRICT.

Alla chiusura logica della relazione:

- valido_al viene valorizzato automaticamente;
- relazione_primaria viene impostata a false.

""

}

```
// -----
// DIPENDENZE DIRETTE DELLA SUPPLY CHAIN
// -----
```

```
Table tipo_dipendenza_servizio {
  id serial [pk]
  codice varchar [unique, note: 'Ad esempio TECNICA o FORNITURA']
  descrizione text
}
```

```
Table servizio_dipendenza_asset {
  servizio_id int [pk, ref: > servizio.id]
  asset_id int [pk, ref: > asset.id]
  tipo_dipendenza_servizio_id int [ref: > tipo_dipendenza_servizio.id]
  descrizione text
}
```

```
Table servizio_dipendenza_fornitore {
  servizio_id int [pk, ref: > servizio.id]
  fornitore_id int [pk, ref: > fornitore.id]
  tipo_dipendenza_servizio_id int [ref: > tipo_dipendenza_servizio.id]
  descrizione text
}
```

```
// -----
// COMPOSIZIONE GERARCHICA DEI SERVIZI
// -----
```

```
Table tipo_dipendenza {
  id serial [pk]
  codice varchar [not null, unique]
  descrizione text
}
```

```
Table esito_impatto {
  id serial [pk]
  codice varchar [not null, unique]
  descrizione text
}
```

```
Table servizio_componente {
  id bigint [pk, increment]

  servizio_padre_id int [
    not null,
    ref: > servizio.id
  ]
```

```
servizio_figlio_id int [  
  not null,  
  ref: > servizio.id  
]
```

```
tipo_dipendenza_id int [ref: > tipo_dipendenza.id]  
esito_impatto_id int [ref: > esito_impatto.id]
```

```
peso_percentuale int [  
  note: 'Valore compreso tra 0 e 100'  
]
```

```
descrizione text
```

```
attiva boolean [  
  not null,  
  default: true  
]
```

```
valido_dal timestampz [  
  not null,  
  default: `now()`  
]
```

```
valido_al timestampz
```

```
relazione_primaria boolean [  
  not null,  
  default: false  
]
```

```
ordine_componente int [  
    not null,  
    default: 1  
]
```

motivo_chiusura text

```
creato_il timestamptz [  
    not null,  
    default: `now()`  
]
```

```
aggiornato_il timestamptz [  
    not null,  
    default: `now()`  
]
```

Note: ""

Relazione direzionale:

servizio_padre_id -> servizio_figlio_id.

Regole implementate in PostgreSQL:

- padre e figlio devono essere differenti;
- peso_percentuale deve essere compreso tra 0 e 100;
- ordine_componente deve essere maggiore di zero;
- una relazione attiva non può avere valido_al;
- una relazione chiusa deve avere valido_al;
- una relazione primaria deve essere attiva;
- una sola relazione attiva per ciascuna coppia padre-figlio;
- un solo padre primario attivo per ogni sottoservizio;
- trigger trg_check_servizio_componente_cycle contro i cicli;
- FK padre e figlio con ON DELETE RESTRICT.

```
'''
```

```
}
```

```
// -----
```

```
// STATO DEI SERVIZI ED EVENTI
```

```
// -----
```

```
Table stato_servizio {
```

```
  id serial [pk]
```

```
  codice varchar [not null, unique]
```

```
  descrizione text
```

```
}
```

```
Table evento_servizio {
```

```
  id serial [pk]
```

```
  servizio_id int [ref: > servizio.id]
```

```
  stato_servizio_id int [ref: > stato_servizio.id]
```

```
  inizio timestamp
```

```
  fine timestamp
```

```
  causa varchar
```

```
  severita varchar [note: 'Bassa, Media, Alta, Critica']
```

```
  segnalatore varchar
```

```
  descrizione_dettagliata text
```

```
  tipologia varchar
```

```
  risoluzione text
```

```
}
```

```
// -----
```

```
// AUDIT LOG
```

```
// -----
```

Table audit_log {

id serial [pk]

asset_id int [

note: 'Nessuna FK intenzionale: il log deve sopravvivere alla rimozione dell asset originario'

]

utente varchar [not null]

operazione varchar [

not null,

note: 'INSERT, UPDATE, DELETE'

]

data_modifica timestamp [default: `now()`]

valore_precedente text

valore_nuovo text

Note: ''

Il trigger attualmente attivo registra:

- operazione;
- utente;
- asset_id;
- timestamp.

valore_precedente e valore_nuovo esistono ma non sono ancora
valorizzati dalla funzione in produzione.

L'assenza della foreign key verso asset è intenzionale:

il registro deve preservare lo storico anche qualora il record

originario non sia più disponibile.

""

}

// -----

// TASSONOMIA INCIDENTI ACN

// -----

Table tassonomia_incidenti_acn {

id serial [pk]

uuid uuid [unique]

macro_area varchar [note: 'BC, TT, TA, AC']

sotto_categoria varchar

codice_acn varchar [unique]

nome_esteso varchar

descrizione text

colore_hex varchar

is_active boolean

created_at timestampz

}

Table evento_tassonomia_acn {

evento_id int [

pk,

ref: > evento_servizio.id

]

tassonomia_id int [

pk,

ref: > tassonomia_incidenti_acn.id

]


```
passo_wizard int [  
    note: 'Valore compreso tra 1 e 6'  
]  
}
```